EN.605.715.81.FA25 Project 8 - Transmitting IMU Data via Arduino and FreeRTOS

Kelly “Scott” Sims

YouTube Link: https://youtu.be/4tFuWsDbgHA
GitHub Link: https://github.com/Mooseburger1/johns_hopkins_masters/tree/main/software_development_for_rt_embedded_systems_EN_605_715_81/project_8

Requirements

Project Key Requirements

1. Using Arduino & FreeRTOS sample IMU data
2. Transmit IMU data to Raspberry Pi over serial
3. Transmit data from Pi to remote client via websockets

Hardware & Protocol Considerations

Microprocessor: Raspberry pi 5
Microcontroller: Arduino Uno R4 WiFi
Communication: WiFi / Serial
OS: Raspbian 64-bit
OS: FreeRTOS
Hardware: Adafruit BNO055 IMU sensor

Design

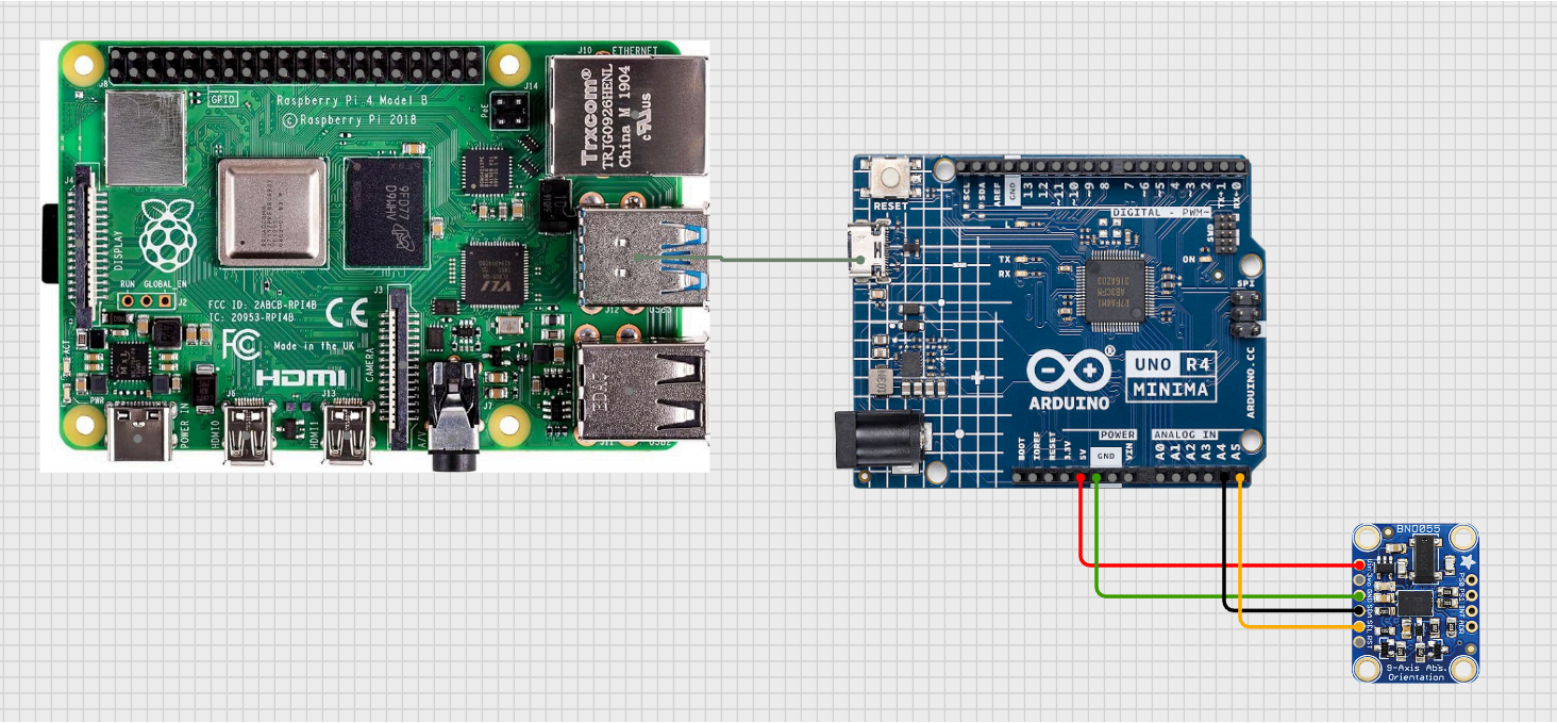
Using an Arduino and FreeRTOS, data will be sampled from a BNO055 IMU sensor using I2C protocol. A task will be created to sample the IMU data. Another task will be scheduled to transmit this data over serial to a Raspberry Pi.

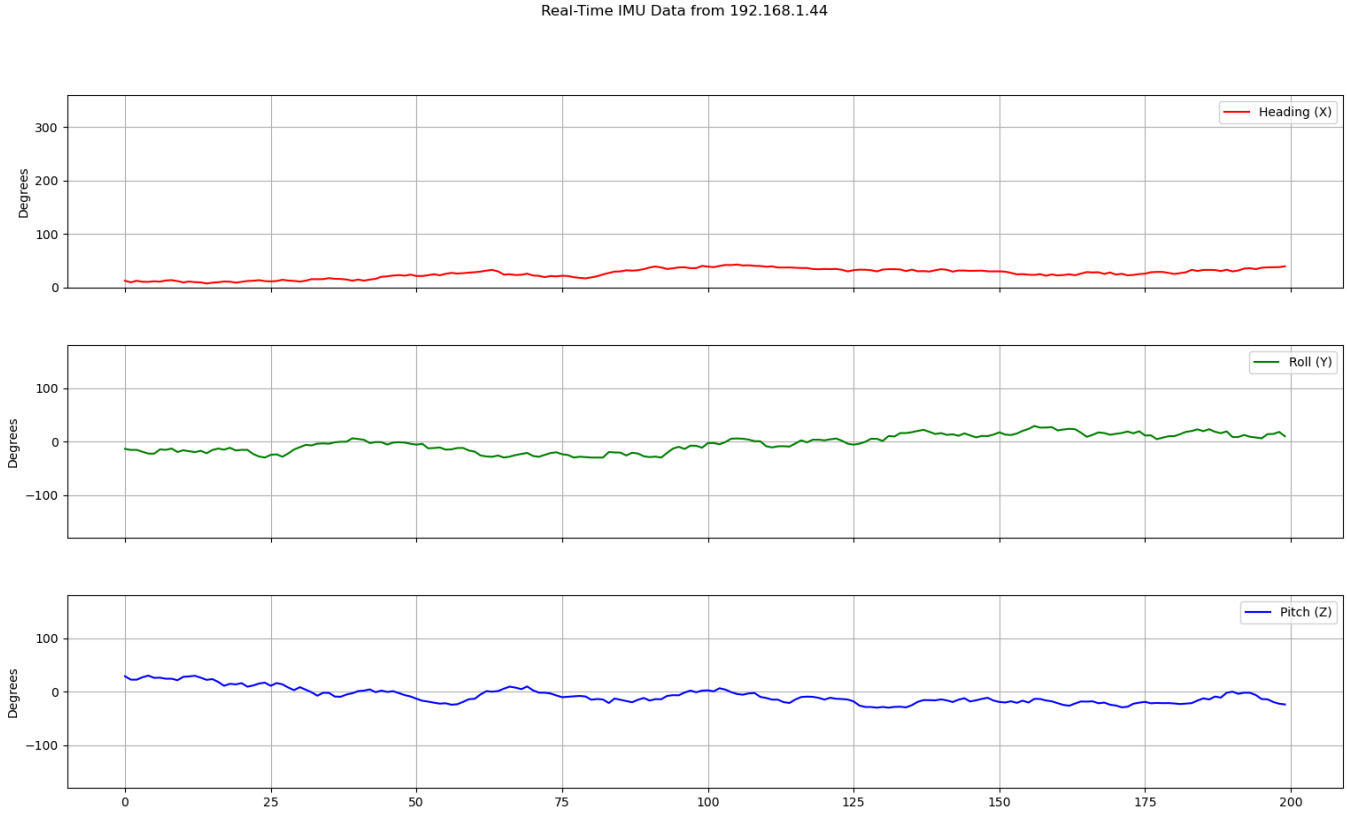
The Raspberry Pi will expose the IMU data via a Websocket server. Any remote client that connects will receive the stream of data in a CSV data format of.

A remote client script will be created to parse the CSV stream transmitted over the websocket connection. It will plot the data in real-time.

x_orienation, y_orientation, z_orienation

Implementation





Arduino + Free RTOS

```
#include <Arduino.h>
#include <Wire.h>
#include <cstdint>
#include <Adafruit_Sensor.h>
#include <Adafruit_BNO055.h>
#include <Arduino_FreeRTOS.h>

Adafruit_BNO055 bno = Adafruit_BNO055(55, 0x28, &Wire);

struct IMUData {
  float orientation_x;
  float orientation_y;
  float orientation_z;
};

QueueHandle_t imuQueue;
TaskHandle_t hSensorTask;
TaskHandle_t hCommsTask;

const uint32_t BAUD_RATE_DEBUG = 115200;
const int QUEUE_LEN = 10;

void TaskSampleIMU(void *pvParameters) {
    vTaskDelay(pdMS_TO_TICKS(100));

    TickType_t xLastWakeTime = xTaskGetTickCount();
    const TickType_t xFrequency = pdMS_TO_TICKS(20); // 50Hz

    for (;;) {
        // Tiny delay to prevent bus congestion
        vTaskDelay(pdMS_TO_TICKS(2));

        sensors_event_t event;
        bno.getEvent(&event);

        IMUData dataPacket;
        dataPacket.orientation_x = event.orientation.x;
        dataPacket.orientation_y = event.orientation.y;
        dataPacket.orientation_z = event.orientation.z;

        // Send to Queue
        xQueueSend(imuQueue, &dataPacket, (TickType_t)0);
```

```
        vTaskDelayUntil(&xLastWakeTime, xFrequency);
    }
}

void TaskTransmit(void *pvParameters) {
  Serial.println(" [Transmit Task] started.");
  IMUData receivedData;

  for (;;) {
    if (xQueueReceive(imuQueue, &receivedData, 2000) == pdPASS) {
      Serial.print(receivedData.orientation_x, 4);
      Serial.print(",");
      Serial.print(receivedData.orientation_y, 4);
      Serial.print(",");
      Serial.println(receivedData.orientation_z, 4);
    }
  }
}

void setup() {
  Serial.begin(BAUD_RATE_DEBUG);

  Serial.println("--- System Booting ---");

    // Wait for BNO hardware to power up
    // The BNO055 needs ~400ms after power-on to be ready for I2C.
    // The Arduino R4 boots in ~100ms. We must wait.
    delay(2000);

    Wire.begin();
    Wire.setClock(100000);

    if (!bno.begin(OPERATION_MODE_CONFIG)) {
        Serial.println("Error: BNO055 init failed. Check Wiring!");
        while (1);
    }

    // 5. Switch to NDOF Mode
    Serial.println("Switching to NDOF Mode...");
    bno.setMode(OPERATION_MODE_NDOF);
    delay(200);

    // 6. Verify Status
    uint8_t system_status, self_test_results, system_error;
    bno.getSystemStatus(&system_status, &self_test_results, &system_error);
```

```
    Serial.print("System Status: "); Serial.print(system_status, HEX);
    Serial.println(" (Target: 5)");
    Serial.print("System Error:  "); Serial.println(system_error, HEX);

    if (system_status == 5) {
        Serial.println("Sensor Locked. Starting Scheduler.");
        imuQueue = xQueueCreate(10, sizeof(IMUData));
        xTaskCreate(TaskSampleIMU, "SampleIMU", 768, NULL, 2, &hSensorTask);
        xTaskCreate(TaskTransmit,  "Transmit",  768, NULL, 1, &hCommsTask);
        vTaskStartScheduler();
    } else {
        Serial.println("FATAL: Sensor still refusing NDOF. Check Voltage (VIN vs 3Vo).");
    }

  imuQueue = xQueueCreate(QUEUE_LEN, sizeof(IMUData));
  if (imuQueue == NULL) {
    Serial.println("Failed to create queue");
    while(1);
  }

  xTaskCreate(
    TaskSampleIMU,
    "SampleIMU",
    600,
    NULL,
    2,
    &hSensorTask
  );

  xTaskCreate(
    TaskTransmit,
    "Transmit",
    600,
    NULL,
    1,
    &hCommsTask
  );

  Serial.println("Starting scheduler");

  vTaskStartScheduler();
}

void loop(){}
```

Raspberry Pi Server

```python
import serial
import socket
import threading
import time
import sys
import random

SERIAL_PORT = '/dev/ttyACM0'
BAUD_RATE = 115200
HOST = '0.0.0.0'
PORT = 65432

latest_data = "No data yet"
data_lock = threading.Lock() # <--- The Mutex
running = True



def generate_dummy_data():
    """Replaces read_serial() for testing without hardware"""
    global latest_data, running

    heading = 0.0
    roll = 0.0
    pitch = 0.0

    print("RUNNING IN DUMMY MODE (No Arduino required)")

    while running:
        # Simulate Drift (Random Walk)
        # Heading rotates slowly (0-360)
        heading = (heading + random.uniform(-1.0, 1.0)) % 360

        # Roll/Pitch wobble around 0 (Clamped to +/- 30 degrees for realism)
        roll = max(-30, min(30, roll + random.uniform(-2.0, 2.0)))
        pitch = max(-30, min(30, pitch + random.uniform(-2.0, 2.0)))

        # 2. Format as CSV matching the BNO055 format
        # "{Heading},{Roll},{Pitch}"
        line = f"{heading:.2f},{roll:.2f},{pitch:.2f}"
```

```
        # 3. Update Shared Memory
        with data_lock:
            latest_data = line

        # 4. Simulate Sample Rate (e.g., 20Hz = 0.05s)
        time.sleep(0.05)

def read_serial():
    """Runs in background thread to keep Serial buffer empty"""
    global latest_data, running

    try:
        ser = serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=1)
        ser.reset_input_buffer()
        print(f"Serial connected on {SERIAL_PORT}")
    except Exception as e:
        print(f"Serial Error: {e}")
        running = False
        return

    while running:
        if ser.in_waiting > 0:
            try:
                line = ser.readline().decode('utf-8').rstrip()
                if line:
                    with data_lock:
                        latest_data = line
            except Exception as e:
                print(f"Serial Read Error: {e}")

def start_server():
    """Main thread: Handles Network Connections"""
    global latest_data, running

    server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)

    try:
        server_socket.bind((HOST, PORT))
        server_socket.listen()
        print(f"📡 Server listening on {HOST}:{PORT}")

        while running:
            conn, addr = server_socket.accept()
            print(f"Connected by {addr}")
```

```
            try:
                while running:
                    with data_lock:
                        current_payload = latest_data

                    message = f"{current_payload}\n"
                    print(message)
                    conn.sendall(message.encode('utf-8'))
                    time.sleep(0.5)
            except (BrokenPipeError, ConnectionResetError):
                print(f"🔌 Client {addr} disconnected.")
            finally:
                conn.close()

    except KeyboardInterrupt:
        running = False
    finally:
        server_socket.close()

if __name__ == "__main__":
    serial_thread = threading.Thread(target=generate_dummy_data, daemon=True)
    # serial_thread = threading.Thread(target=read_serial, daemon=True)
    serial_thread.start()
    start_server()
```

Remote Client

```
import socket
import threading
import matplotlib.pyplot as plt
import matplotlib.animation as animation
from collections import deque


PI_IP = '192.168.1.44'
PORT = 65432
MAX_POINTS = 200

data_x = deque([0.0] * MAX_POINTS, maxlen=MAX_POINTS)
data_y = deque([0.0] * MAX_POINTS, maxlen=MAX_POINTS)
data_z = deque([0.0] * MAX_POINTS, maxlen=MAX_POINTS)

running = True
```

```
def data_listener():
    """Background thread to fetch data from Pi without blocking the plot"""
    global running
    print(f"Attempting to connect to {PI_IP}:{PORT}...")

    while running:
        try:
            with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
                s.connect((PI_IP, PORT))
                s.setsockopt(socket.IPPROTO_TCP, socket.TCP_NODELAY, 1)

                print("Connected! Streaming data...")
                buffer = ""

                while running:
                    try:
                        chunk = s.recv(1024).decode('utf-8')
                        print(chunk)
                        if not chunk: break

                        buffer += chunk
                        while '\n' in buffer:
                            line, buffer = buffer.split('\n', 1)
                            if "," in line:
                                try:
                                    # Parse CSV: Heading, Roll, Pitch
                                    parts = line.split(',')
                                    if len(parts) == 3:
                                        val_x = float(parts[0])
                                        val_y = float(parts[1])
                                        val_z = float(parts[2])

                                        # Update shared buffers
                                        data_x.append(val_x)
                                        data_y.append(val_y)
                                        data_z.append(val_z)
                                except ValueError:
                                    pass
                    except socket.error:
                        break
        except (ConnectionRefusedError, TimeoutError):
            print("Connection failed. Retrying in 1s...")
            import time; time.sleep(1)
        except Exception as e:
            print(f"Error: {e}")
            break
```

```
# SETUP PLOTS
fig, (ax1, ax2, ax3) = plt.subplots(3, 1, sharex=True, figsize=(8, 8))
plt.subplots_adjust(hspace=0.3)
fig.suptitle(f'Real-Time IMU Data from {PI_IP}')

# Heading Plot
line1, = ax1.plot(list(data_x), label='Heading (X)', color='r')
ax1.set_ylabel('Degrees')
ax1.set_ylim(0, 360)
ax1.legend(loc='upper right')
ax1.grid(True)

# Roll Plot
line2, = ax2.plot(list(data_y), label='Roll (Y)', color='g')
ax2.set_ylabel('Degrees')
ax2.set_ylim(-180, 180)
ax2.legend(loc='upper right')
ax2.grid(True)

# Pitch Plot
line3, = ax3.plot(list(data_z), label='Pitch (Z)', color='b')
ax3.set_ylabel('Degrees')
ax3.set_ylim(-180, 180)
ax3.legend(loc='upper right')
ax3.grid(True)

def update_plot(frame):
    """Called periodically by FuncAnimation to update lines"""
    line1.set_ydata(list(data_x))
    line2.set_ydata(list(data_y))
    line3.set_ydata(list(data_z))
    return line1, line2, line3


t = threading.Thread(target=data_listener, daemon=True)
t.start()

# Start Animation (Interval is in milliseconds)
# 20ms = 50 FPS refresh rate
ani = animation.FuncAnimation(fig, update_plot, interval=20, blit=False)

print("Close the plot window to exit.")
plt.show()
running = False
```